Manual básico de GitHub

Unidad 2. Entornos de Desarrollo, 1.º DAW

Índice

Para qué sirve esto **2**

Parte 0. Traerte el material de clase **4**
- 0.1. Por qué no vale con descargar un ZIP . . . 4
- 0.2. Instalar Git . . . 4
- 0.3. Prepara tus dos carpetas . . . 4
- 0.4. Clonar el repositorio . . . 5
- 0.5. Traerte lo nuevo: pull . . . 5
- 0.6. La norma más importante de esta parte . . . 5
- 0.7. Si el pull te da error . . . 5
- 0.8. Problemas típicos de esta parte . . . 6
- 0.9. Resumen de la Parte 0 . . . 6

Parte 1. La cuenta **7**
- 1.1. Git y GitHub no son lo mismo . . . 7
- 1.2. Crear la cuenta . . . 7
- 1.3. Activar la verificación en dos pasos . . . 8
- 1.4. Dejar el perfil presentable . . . 8

Parte 2. Tu primer repositorio **9**
- 2.1. Qué es un repositorio . . . 9
- 2.2. Crearlo . . . 9
- 2.3. Las tres cosas que acabas de crear . . . 9
- 2.4. La pantalla del repositorio, parte por parte . . . 9
- 2.5. Escribir el README . . . 10
- 2.6. Editar un fichero desde la web . . . 11
- 2.7. Qué es un commit . . . 11
- 2.8. Subir ficheros que ya tienes . . . 12
- 2.9. Crear carpetas . . . 12
- 2.10. Borrar y renombrar . . . 12
- 2.11. Ver el historial . . . 12

Parte 3. Conectar con tu ordenador **13**
- 3.1. El ciclo de trabajo . . . 13
- 3.2. Lo más importante de esta parte: hay varias copias . . . 13
- 3.3. Instalar Git . . . 14
- 3.4. Decirle a Git quién eres . . . 14
- 3.5. Traer el repositorio a tu ordenador . . . 14
- 3.6. El día a día en VS Code . . . 15
- 3.7. Antes de empezar a trabajar, pull . . . 16

3.8. Ver los cambios antes de confirmarlos . . . 16
3.9. Si prefieres una aplicación aparte: GitHub Desktop . . . 16
3.10. Lo que no hay que hacer . . . 17

Parte 4. Cosas útiles del día a día **18**
4.1. Publicar una página web gratis . . . 18
4.2. Issues: la lista de tareas del proyecto . . . 18
4.3. Añadir a alguien a tu repositorio . . . 18
4.4. Buscar y guardar proyectos de otros . . . 18
4.5. Tu perfil como tarjeta de visita . . . 19

Parte 5. Cuando algo va mal **20**

Lista de comprobación **21**

Glosario **22**

Resumen en diez líneas **22**

Para qué sirve esto

Este manual te lleva de la mano desde no tener cuenta hasta tener tu código guardado en internet, con su historial de cambios, y publicado en una página web si quieres.

No hace falta saber nada previo. Cada apartado es corto y va en orden: si los sigues seguidos, al final tienes todo montado y funcionando.

Parte	Qué vas a conseguir
0	**Traerte el material de clase y mantenerlo al día. Empieza por aquí**
1	Tener la cuenta creada, segura y con tu perfil presentable
2	Tu primer repositorio, trabajando desde el navegador
3	Conectar GitHub con tu ordenador y con VS Code
4	Publicar una página web gratis y usar las herramientas del día a día
5	Salir de los líos más típicos

La **Parte 0** se hace la primera y es independiente del resto: no necesita cuenta de GitHub ni nada instalado salvo Git. Las demás van en orden.

Todo está en inglés y no se traduce. Los botones de GitHub, los de VS Code y los comandos usan siempre las mismas seis o siete palabras. Si te las aprendes ahora, el resto del manual se lee solo:

Palabra	Se pronuncia	Significa
repository	*ripósitori*	El proyecto con todo su historial. Se dice «repo»
commit	*comít*	Guardar una foto del proyecto, con su explicación
push	*push*	Empujar: mandar tus commits a GitHub

Palabra	Se pronuncia	Significa
pull	*pul*	Tirar: traerte a tu ordenador lo que hay en GitHub
clone	*clon*	Descargar un repositorio entero por primera vez
branch	*branch*	Rama: una línea de trabajo paralela
issue	*íshu*	Una tarea pendiente o un fallo anotado
merge	*merch*	Fusionar: juntar dos líneas de trabajo

No intentes traducirlas al hablar. En esta profesión se dice «haz un commit» y «súbelo con push», y así es como te vas a entender con cualquiera.

Parte 0. Traerte el material de clase

Esto es lo primero y lo vas a usar todos los días. El material de la asignatura está en un repositorio público de GitHub, y va creciendo: cada clase se sube algo nuevo.

Tu trabajo aquí es **traértelo una vez y ponerlo al día cuando haga falta**. Son dos minutos el primer día y diez segundos cada día siguiente.

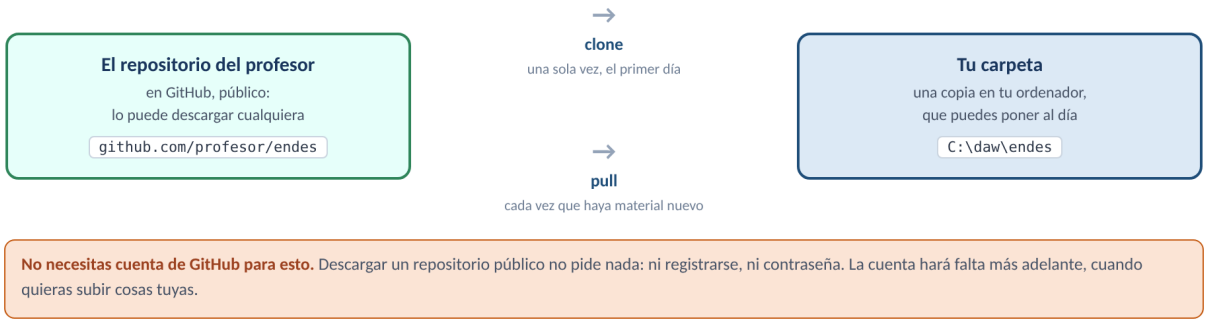


Figura 1: *Lo traes una vez con clone y lo actualizas con pull*

Idea clave

No necesitas cuenta de GitHub para esta parte. El repositorio es público: se descarga sin registrarse y sin contraseña. La cuenta hace falta a partir de la Parte 1, cuando quieras tener cosas tuyas ahí.

0.1. Por qué no vale con descargar un ZIP

En GitHub hay un botón **Code > Download ZIP** que baja todo de golpe. Funciona, pero es un callejón sin salida: cuando se suba material nuevo tendrás que volver a descargar el ZIP entero, descomprimirlo otra vez y acordarte de cuál era el viejo.

Clonando, no. Clonar deja una carpeta conectada con la de GitHub, y para ponerla al día basta con pulsar un botón. Por eso se hace así.

0.2. Instalar Git

Git es un programa aparte de VS Code. Hay que instalarlo una vez en cada ordenador donde trabajes.

Windows: descarga de `git-scm.com` y acepta todas las opciones por defecto. **Linux:** `sudo apt install git`

Para comprobar que está, abre una terminal (en VS Code, menú **Terminal > Nueva terminal**, o `Ctrl+ñ`) y escribe:

```
git --version
```

Si contesta algo como `git version 2.43.0`, listo. Si dice que no reconoce el comando, cierra la terminal, vuelve a abrirla y prueba otra vez; si sigue igual, es que no se ha instalado.

0.3. Prepara tus dos carpetas

Antes de descargar nada, crea esta estructura en tu disco. **Es importante y luego se ve por qué:**

```
C:\daw\
   ├── material\          <- aquí va lo del profesor. NO se toca
   └── mis-ejercicios\    <- aquí trabajas tú
```

En Linux, lo mismo colgando de tu carpeta personal: `~/daw/material` y `~/daw/mis-ejercicios`.

0.4. Clonar el repositorio

1. Entra en la dirección del repositorio de la asignatura. **La tienes en Moodle**, y tiene esta pinta: `github.com/nombre-del-profesor/nombre-del-repositorio`.
2. Pulsa el botón verde **Code**, pestaña **HTTPS**, y copia la dirección con el icono de copiar. Termina en `.git`.
3. En VS Code, pulsa `Ctrl+Mayús+P`, escribe **Git: Clone** y pulsa Intro.
4. Pega la dirección y pulsa Intro.
5. Te pregunta en qué carpeta guardarlo: elige **`C:\daw\material`**.
6. Cuando termine, VS Code te pregunta si quieres abrir la carpeta. Di que sí.

Ya tienes todo el material en tu ordenador. Ábrelo y míralo: son ficheros normales.

Para el examen / Ojo

Si te pide usuario y contraseña, algo va mal. Un repositorio público no pide nada. Lo más probable es que hayas copiado mal la dirección, o que hayas copiado la de la barra del navegador en vez de la del botón **Code**.

0.5. Traerte lo nuevo: pull

Esta es la parte que vas a repetir todo el curso. Cuando se anuncie que hay material nuevo:

1. Abre en VS Code la carpeta `material` (**Archivo > Abrir carpeta**).
2. Abajo a la izquierda, en la barra azul, pulsa el **icono de las dos flechas en círculo**. También vale `Ctrl+Mayús+P` > **Git: Pull**.
3. En dos segundos aparecen los ficheros nuevos en el explorador de la izquierda.

Y ya está. No hay que descargar nada, ni descomprimir, ni mirar fechas: **lo que tengas en esa carpeta es exactamente lo que hay publicado**.

Idea clave

Hazlo al empezar cada clase. Diez segundos. Si alguna vez parece que te falta un fichero que los demás sí tienen, lo primero que hay que probar es un pull.

0.6. La norma más importante de esta parte

Para el examen / Ojo

No escribas nada dentro de la carpeta `material`. Ni los ejercicios, ni apuntes, ni pruebas.
Esa carpeta es un espejo de la del profesor. Si cambias un fichero que luego él también cambia, el `pull` se para y te suelta un error raro sobre conflictos, justo el día que tienes prisa.
Para hacer un ejercicio: copia la carpeta de ese ejercicio a `mis-ejercicios` y trabaja en la copia. Arrastrando con el ratón, como cualquier carpeta.

`C:\daw\material`	Se lee. Se actualiza con pull. **No se escribe**
`C:\daw\mis-ejercicios`	Tuya. Aquí copias, rompes, pruebas y entregas

0.7. Si el pull te da error

Pasa cuando has tocado algo de la carpeta `material` sin darte cuenta: abriste un fichero, le diste a guardar y ya está.

Como ahí dentro no tienes nada tuyo que perder, la solución es **descartar tus cambios** y volver a dejar la carpeta como está en GitHub:

1. Abre el panel de **Control de código fuente** (`Ctrl+Mayús+G`).
2. Verás los ficheros que has tocado.
3. En la cabecera de **Cambios**, pulsa el icono de **deshacer**, el de la flecha curva: *Discard All Changes*.
4. Confirma, y vuelve a hacer pull.

Para el examen / Ojo

Ese botón **borra los cambios y no hay vuelta atrás**. Úsalo solo en la carpeta `material`, donde nada es tuyo. En tus propias carpetas, nunca sin mirar antes qué vas a perder.

0.8. Problemas típicos de esta parte

Qué pasa	Por qué	Qué hacer
«git no se reconoce como comando»	No está instalado, o la terminal es anterior a la instalación	Cierra la terminal y ábrela otra vez; si sigue, reinstala Git
Me pide usuario y contraseña	Dirección mal copiada	Cópiala del botón verde **Code**, pestaña HTTPS
El pull no trae nada	No hay material nuevo todavía	Nada que hacer: ya lo tienes todo
«Your local changes would be overwritten»	Has escrito dentro de `material`	Apartado 0.7
No encuentro la carpeta clonada	La guardaste en otro sitio	`Ctrl+Mayús+P` > *Git: Clone* otra vez, eligiendo bien la carpeta

0.9. Resumen de la Parte 0

1. Instalas Git **una vez**.
2. Clonas el repositorio **una vez**, en `C:\daw\material`.
3. Haces **pull** al empezar cada clase.
4. **No escribes nunca** dentro de esa carpeta: copias a `mis-ejercicios` y trabajas ahí.

Con esto ya puedes seguir la asignatura. El resto del manual es para cuando quieras tener tus propios proyectos en GitHub, y empieza en la página siguiente.

Parte 1. La cuenta

1.1. Git y GitHub no son lo mismo

Antes de nada, aclara esto, porque todo lo demás depende de ello.

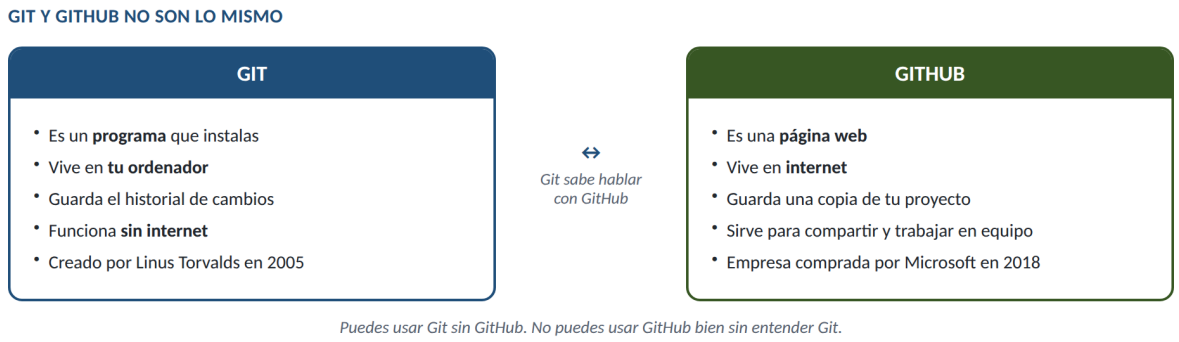


Figura 2: *Git trabaja en tu ordenador; GitHub vive en Internet*

Git es un programa que se instala en tu ordenador y lleva la cuenta de todos los cambios que haces en un proyecto. Funciona sin internet.

GitHub es una página web donde puedes guardar una copia de ese proyecto para tenerlo a salvo, verlo desde cualquier sitio y compartirlo con otras personas.

Hay más sitios parecidos a GitHub —GitLab, Bitbucket, Codeberg— y todos usan Git por debajo. GitHub es el más usado con diferencia.

1.2. Crear la cuenta

1. Entra en **github.com** y pulsa **Sign up**.
2. **Correo electrónico.** Usa uno al que vayas a tener acceso siempre. Si usas el del instituto, ten en cuenta que lo perderás al terminar el ciclo; puedes cambiarlo más adelante.
3. **Contraseña.** Mínimo 15 caracteres, o bien 8 con números y minúsculas. Usa una que no uses en ningún otro sitio.
4. **Nombre de usuario.** Piénsalo dos segundos, que es lo único de aquí que cuesta cambiar:

Para el examen / Ojo

Tu nombre de usuario de GitHub es tu identidad profesional. Va a aparecer en la dirección de todos tus proyectos (`github.com/tunombre`), lo vas a poner en el currículum y lo va a ver quien te entreviste para unas prácticas.
Sirve: `avazquez`, `antonio-vazquez`, `avazquezdev`, `antoniovc`
No sirve: `xXDarkCoderXx`, `elputoamo99`, `asdasd123`. Parecen graciosos ahora. Dentro de dos años, cuando lo tengas que poner en una solicitud de prácticas, no lo son.
Se puede cambiar después, pero se rompen todos los enlaces que hayas compartido.

5. **Resuelve el puzle** de verificación. A veces pone varios seguidos.
6. **Código de verificación.** Te llega al correo. Cópialo y pégalo.
7. Te preguntará cuántas personas sois y para qué lo vas a usar. Puedes **saltarte** esas preguntas sin problema.
8. Cuando te ofrezca planes, elige **Free**. El gratuito incluye repositorios ilimitados, tanto públicos como privados.

1.3. Activar la verificación en dos pasos

Esto no es opcional. GitHub exige la verificación en dos pasos para poder seguir usando la cuenta, así que es mejor hacerlo ahora con calma que cuando te eche y tengas una entrega pendiente.

1. Arriba a la derecha, pulsa tu foto > **Settings**.
2. En el menú de la izquierda: **Password and authentication**.
3. **Enable two-factor authentication**.

Te dará a elegir cómo:

Método	Qué es	Recomendación
Aplicación de códigos	Una app del móvil que genera un número cada 30 segundos	**La mejor**
Llave de seguridad	Un aparatito USB	Si tienes uno
SMS	Un mensaje al móvil	La peor, pero mejor que nada

Para la primera: instala en el móvil **Google Authenticator**, **Microsoft Authenticator** o **Aegis**. Abre la app, escanea el código QR que te enseña GitHub y escribe el número de seis cifras que aparece.

Para el examen / Ojo

Guarda los códigos de recuperación. Después de activarlo, GitHub te enseña una lista de códigos de un solo uso. **Descárgalos y guárdalos en un sitio que no sea tu móvil.**
Son la única forma de entrar si pierdes el teléfono. Sin ellos y sin el móvil, la cuenta se pierde y no hay nadie a quien reclamar. Pasa todos los cursos.

1.4. Dejar el perfil presentable

Cinco minutos que valen la pena. En **Settings > Public profile**:

Campo	Qué poner
Name	Tu nombre y apellidos reales
Bio	Una línea: «Estudiante de DAW. Aprendiendo Python y desarrollo web»
Photo	Una foto normal o un avatar. El gato por defecto no dice nada bueno
Location	Tu ciudad o provincia

Parte 2. Tu primer repositorio

2.1. Qué es un repositorio

Un **repositorio** —«repo», en la jerga— es **una carpeta de proyecto con memoria**.

Guarda tus ficheros, igual que una carpeta normal, pero además guarda **todas las versiones por las que ha pasado cada uno**: quién lo cambió, cuándo y qué cambió exactamente. Y nunca se pierde nada.

Un repositorio por proyecto. No metas todas las prácticas del curso en uno solo.

2.2. Crearlo

1. Arriba a la derecha, el botón **+** > **New repository**.
2. Rellena:

Campo	Qué poner
Repository name	Sin espacios ni acentos. Guiones en lugar de espacios: `calculadora-python`
Description	Una frase. Opcional, pero se ve en tu perfil
Public / Private	Ver el recuadro de abajo
Add a README file	**Márcalo siempre**
Add .gitignore	Elige la plantilla del lenguaje que vayas a usar
Choose a license	Para prácticas de clase puedes dejarlo en *None*

3. **Create repository**.

Idea clave

¿Público o privado?

- **Público**: lo ve todo el mundo. Es lo normal para prácticas, proyectos personales y cualquier cosa que quieras enseñar. Lo que publicas aquí es tu escaparate.
- **Privado**: solo lo ves tú y quien tú invites. Úsalo si el proyecto lleva datos personales, contraseñas o algo que no deba salir.

Los dos son gratis e ilimitados. **Si dudas, empieza en privado**: pasar de privado a público es un clic; al revés, lo que ya se ha visto, visto está.

2.3. Las tres cosas que acabas de crear

README.md es la portada del proyecto. GitHub lo muestra automáticamente debajo de la lista de ficheros. Si alguien entra en tu repositorio, esto es lo primero que lee.

.gitignore es una lista de ficheros que **no** quieres subir: las carpetas que genera el editor, los ficheros temporales, las contraseñas. Si eliges la plantilla de Python o de Node, ya viene rellena con lo habitual.

LICENSE dice qué puede hacer otra gente con tu código. Si no pones ninguna, por defecto **nadie puede usarlo legalmente** aunque esté publicado.

2.4. La pantalla del repositorio, parte por parte

Antes de tocar nada, conviene saber qué es cada cosa. Esta es la pantalla que vas a ver mil veces este curso:

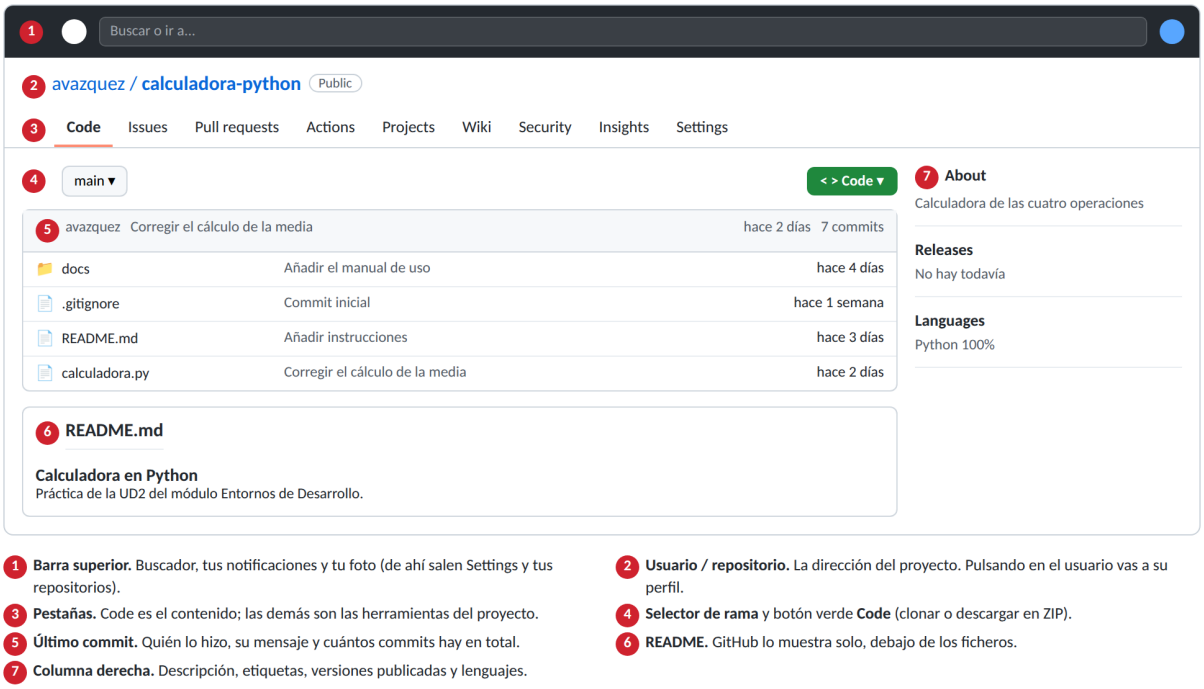


Figura 3: *Las siete zonas de la página de un repositorio*

Las **pestañas** de arriba son las herramientas del proyecto. De momento solo necesitas tres:

Pestaña	Qué hay dentro
Code	Los ficheros, las carpetas y el historial. Es donde vas a estar casi siempre
Issues	La lista de tareas pendientes y de fallos por corregir
Settings	La configuración: colaboradores, GitHub Pages, borrar el repositorio

Las otras (*Pull requests*, *Actions*, *Projects*, *Wiki*, *Security*, *Insights*) son de trabajo en equipo y de automatización. Se ven en el manual avanzado.

Idea clave

Todo esto se hace con el ratón. No hay que escribir ni un comando para crear un repositorio, subir ficheros, editarlos o ver el historial. Los comandos de terminal aparecen en la Parte 3, y aun ahí son solo dos, de una vez para siempre.

2.5. Escribir el README

El README se escribe en **Markdown**, que es texto normal con unas pocas marcas. Es lo mismo que usarás en mil sitios más.

Escribes	Sale
`# Título`	Un título grande
`## Subtítulo`	Un título más pequeño
`**negrita**`	**negrita**
`*cursiva*`	*cursiva*

Escribes	Sale
`- Elemento`	Un punto de lista
`1. Elemento`	Una lista numerada
`` `codigo` ``	`código` dentro de una frase
`[texto](enlace)`	Un enlace
`![](imagen.png)`	Una imagen

Para un bloque de código entero, tres acentos graves, el nombre del lenguaje, el código y tres acentos graves otra vez.

Un README decente para una práctica tiene esto:

```
# Calculadora en Python

Práctica de la UD2 del módulo Entornos de Desarrollo.

## Qué hace

Pide dos números y muestra el resultado de las cuatro operaciones.

## Cómo se usa

1. Descarga el fichero `calculadora.py`
2. Ejecuta `python calculadora.py`

## Autor

Nombre y apellidos
```

Dos cosas que descolocan la primera vez:

- **Un salto de línea no hace salto de línea.** Si escribes dos frases en líneas seguidas, Markdown las junta en un mismo párrafo. Para separarlas de verdad, **deja una línea en blanco** entre ellas.
- **Se escribe con marcas, pero se lee formateado.** Mientras editas ves `**negrita**`; el resultado solo se ve al guardar. En GitHub tienes la pestaña **Preview** encima del recuadro de edición, y en VS Code se previsualiza con `Ctrl+Mayús+V`. Úsalo antes de dar por bueno el README.

2.6. Editar un fichero desde la web

1. Pulsa sobre el fichero.
2. El icono del **lápiz**, arriba a la derecha.
3. Cambia lo que quieras.
4. Baja hasta el recuadro **Commit changes**.

2.7. Qué es un commit

Un **commit** es **una foto de tu proyecto en un momento dado**, con un mensaje que explica qué has cambiado.

Cada vez que haces un commit, GitHub guarda ese estado para siempre. Puedes volver a él dentro de un año.

Para el examen / Ojo

El mensaje del commit importa más de lo que parece. Es lo que vas a leer dentro de seis meses intentando entender qué hiciste.
Mensajes buenos: `Añadir validación del correo`, `Corregir el cálculo de la media`, `Crear la página de contacto`.
Mensajes inútiles: `cambios`, `asdf`, `update`, `ya funciona`.
Escríbelo **en imperativo y diciendo qué hace el cambio**, no qué has tocado. Son cinco segundos y es la diferencia entre un historial que sirve y uno que no.

2.8. Subir ficheros que ya tienes

1. Botón **Add file** > **Upload files**.
2. **Arrastra** los ficheros o la carpeta entera desde tu ordenador.
3. Escribe el mensaje del commit y confirma.

2.9. Crear carpetas

GitHub no tiene botón de «nueva carpeta». Se hace con un truco:

1. **Add file** > **Create new file**.
2. En el nombre, escribe `docs/notas.md`.
3. **En cuanto escribes la barra `/`, GitHub crea la carpeta** y te deja dentro.

Para carpetas más profundas, más barras: `src/python/main.py`.

2.10. Borrar y renombrar

Abre el fichero y, arriba a la derecha, el menú de los **tres puntos**: ahí tienes *Rename* y *Delete file*. Las dos cosas son un commit más, así que quedan en el historial y se pueden deshacer.

2.11. Ver el historial

Esta es la razón de ser de todo esto.

Dónde	Qué ves
Pestaña **Commits**	La lista de todos los cambios, con fecha y autor
Pulsando en un commit	Exactamente qué líneas cambiaron: en **verde** lo añadido, en **rojo** lo quitado
Botón **History** de un fichero	Solo los cambios de ese fichero
Botón **Blame**	Quién escribió cada línea y en qué commit

Idea clave

Blame es el botón más útil y el peor nombrado. Sirve para, ante una línea rara, ver de qué commit salió y leer el mensaje de quien la escribió. En un proyecto en equipo lo vas a usar constantemente, y no es para echar culpas: es para entender por qué algo está como está.

Parte 3. Conectar con tu ordenador

Trabajar desde la web está bien para cambios sueltos, pero lo normal es trabajar en tu ordenador con VS Code y sincronizar.

3.1. El ciclo de trabajo

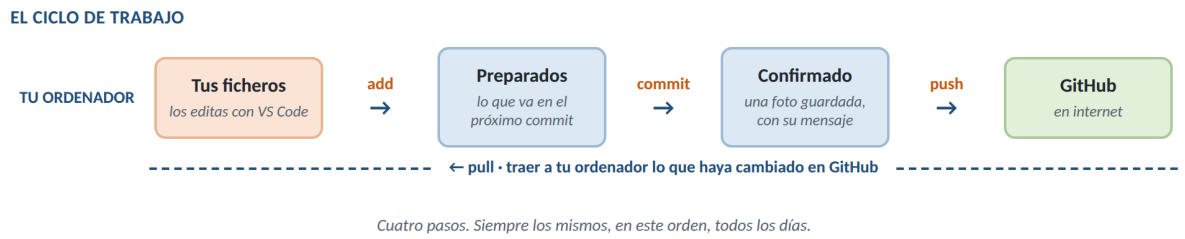


Figura 4: *El ciclo de trabajo con Git y GitHub*

Son cuatro pasos y siempre los mismos:

1. **Editas** tus ficheros normalmente.
2. **Preparas** los cambios que quieres guardar (en inglés, *stage*).
3. **Confirmas** con un commit y su mensaje.
4. **Subes** a GitHub con *push*.

Y en sentido contrario, **pull** trae a tu ordenador lo que haya cambiado en GitHub.

3.2. Lo más importante de esta parte: hay varias copias

Esto es lo que más líos causa durante todo el curso, así que léelo dos veces.

Cuando te traes el proyecto a tu ordenador, **no estás abriendo el de GitHub**: estás haciendo una copia. A partir de ese momento hay dos proyectos distintos, cada uno con su historial, y **nadie los sincroniza por ti**.



Figura 5: *Cada sitio tiene su propia copia; push y pull son los únicos puentes*

A la copia que está en GitHub, Git la llama **`origin`**. Ese nombre te va a aparecer en botones y mensajes de error, y significa simplemente «el GitHub de este proyecto».

De aquí salen las dos costumbres que te van a ahorrar la mitad de los problemas del curso:

Cuándo	Qué haces	Por qué
Al sentarte a trabajar	**pull**	Traerte lo que hayas hecho en otro sitio

Cuándo	Qué haces	Por qué
Al levantarte, aunque sea a medias	**push**	Que no se quede solo en ese ordenador

Para el examen / Ojo

El caso que pasa todos los cursos: trabajas en el aula, subes, llegas a casa, abres el proyecto en tu portátil **sin hacer pull** y sigues editando. Ahora tienes dos versiones distintas que han avanzado por su cuenta, y al intentar subir te sale un error. Se arregla, pero se tarda más que los dos segundos del pull.

3.3. Instalar Git

Git es un programa aparte de VS Code, y hay que instalarlo una vez en cada ordenador donde vayas a trabajar.

Windows: descarga de `git-scm.com` y acepta todas las opciones por defecto. **Linux:** `sudo apt install git` **macOS:** suele venir; si no, se instala con las herramientas de desarrollo.

Para comprobar que ha quedado bien hay que abrir una **terminal**, que es una ventana donde se escriben órdenes en vez de pulsar botones:

Sistema	Cómo se abre
Windows	Tecla Windows, escribe `cmd` y pulsa Intro
Linux	`Ctrl+Alt+T`
Desde VS Code	Menú **Terminal > Nueva terminal**, o `Ctrl+ñ`. Es la más cómoda

En la ventana que se abre, escribe esto y pulsa Intro:

```
git --version
```

Si contesta algo como `git version 2.43.0`, está instalado. Si dice que no se reconoce el comando, no lo está, o hay que cerrar y volver a abrir la terminal para que se entere.

Idea clave

No te asustes con la terminal: en todo este manual se usa **dos veces**, aquí y en el apartado siguiente, y después no vuelve a aparecer. Todo lo demás se hace con el ratón.

3.4. Decirle a Git quién eres

Una sola vez en la vida, en la terminal. Git necesita saber con qué nombre firmar tus commits:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tucorreo@ejemplo.com"
```

Usa **el mismo correo que en GitHub**. Si no, tus commits aparecerán como de un desconocido.

3.5. Traer el repositorio a tu ordenador

1. En la página de tu repositorio, botón verde **Code** > copia la dirección **HTTPS**.
2. En VS Code, `Ctrl+Mayús+P` > escribe **Git: Clone** > pega la dirección.
3. Elige en qué carpeta quieres guardarlo.
4. Cuando termine, VS Code te pregunta si quieres abrirlo. Di que sí.

La primera vez te pedirá entrar en GitHub. Se abrirá el navegador: autoriza y listo. A eso se le llama **clonar**.

Qué acaba de aparecer en tu disco. Una carpeta con el nombre del repositorio, con tus ficheros dentro y una carpeta oculta llamada `.git`. Ahí está guardado todo el historial: cada commit, cada versión de cada fichero.

Para el examen / Ojo

Tres reglas sobre esa carpeta:

1. **No toques ni borres `.git`.** Si la borras, la carpeta deja de ser un repositorio: se queda en ficheros sueltos, sin historial y sin conexión con GitHub.
2. **No muevas la carpeta a otro sitio mientras VS Code la tiene abierta.** Ciérralo antes.
3. **No clones un repositorio dentro de otro repositorio.** Si lo haces, Git se lía y tú también. Ten una carpeta tipo daw y clona ahí dentro, uno al lado de otro.

Y un despiste muy frecuente: en VS Code tienes que abrir **la carpeta del proyecto**, no la que la contiene. Si abres la de arriba, el panel de Control de código fuente aparece vacío y parece que Git no funciona. Mira el título de la ventana: tiene que poner el nombre del repositorio.

3.6. El día a día en VS Code

En la barra de iconos de la izquierda hay uno de **tres puntos unidos por líneas**: es el **Control de código fuente** (`Ctrl+Mayús+G`). Todo lo que viene a partir de aquí se hace pulsando botones, no escribiendo comandos.

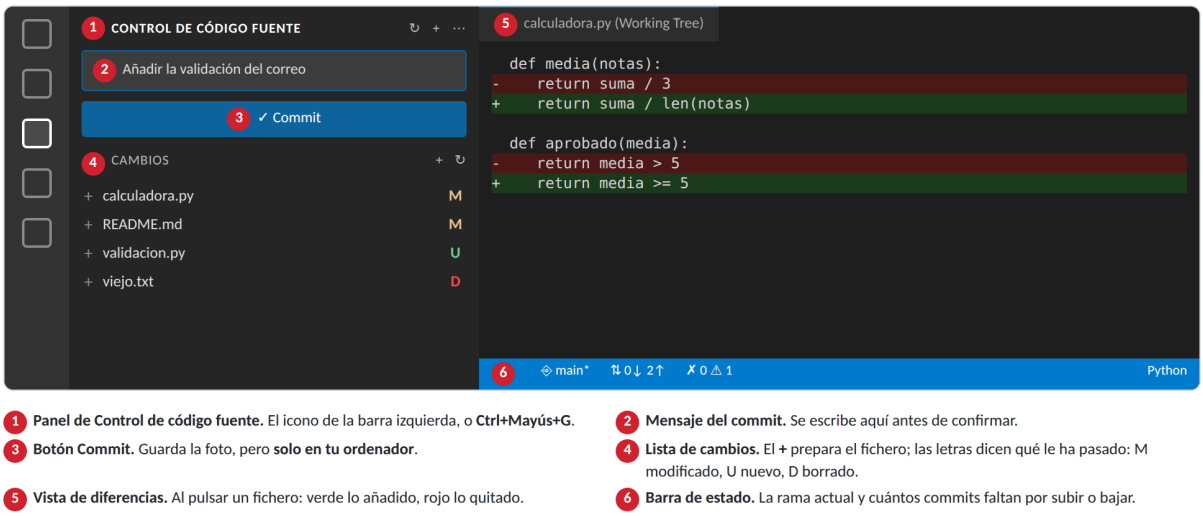


Figura 6: *El panel de Control de código fuente de VS Code*

Ahí aparecen todos los ficheros que has tocado desde el último commit, cada uno con una letra:

Marca	Significa
U	Fichero nuevo, sin seguimiento (*untracked*)
M	Modificado
D	Borrado

Por qué hay que «preparar» antes de confirmar. Es el paso que más raro se hace al principio. Piénsalo como una caja de mudanza: vas metiendo en la caja lo que quieres mandar (**preparar**) y, cuando está lo

que tiene que estar, la cierras y le pones una etiqueta (**commit**). Lo que no metas en la caja se queda en tu carpeta, sin confirmar, esperando al próximo commit.

Existe para que puedas confirmar **solo una parte** de lo que has tocado. Si has arreglado un fallo y además has dejado a medias otra cosa, preparas solo el arreglo y haces un commit limpio.

Para guardar los cambios:

1. Pasa el ratón por cada fichero y pulsa el **+** para prepararlo. *(O el + de la cabecera para preparar todos.)*
2. Escribe el **mensaje** en el recuadro de arriba.
3. Pulsa **Commit**.
4. Pulsa **Sync Changes** (o el botón de las flechas de la barra de abajo) para subirlo.

La barra azul de abajo del todo merece un vistazo: a la izquierda pone el nombre de la rama en la que estás (`main`) y, al lado, dos números con flechas. El de la flecha hacia arriba son los commits que has hecho y **todavía no has subido**. Si ese número no es cero, te falta el push.

Para el examen / Ojo

Commit no es subir. Son dos cosas distintas y es la confusión número uno:

- **Commit** guarda la foto **en tu ordenador**. Puedes hacer diez commits sin internet.
- **Push** manda esos commits **a GitHub**.

Si solo haces commit y cierras el portátil, en GitHub no hay nada. Si el portátil se rompe, perdiste el trabajo. **Haz push al terminar de trabajar, siempre.**

3.7. Antes de empezar a trabajar, pull

Si has tocado algo desde la web, o si trabajas en dos ordenadores, **lo primero al sentarte es traer los cambios**: el botón de sincronizar, o `Ctrl+Mayús+P` > **Git: Pull**.

Hacer pull antes de empezar evita la mayoría de los líos.

3.8. Ver los cambios antes de confirmarlos

En el panel de Control de código fuente, **pulsa sobre un fichero modificado**: VS Code abre una vista a dos columnas con el antes y el después, y las líneas cambiadas resaltadas.

Míralo siempre antes de hacer commit. Se pillan ahí muchos despistes.

3.9. Si prefieres una aplicación aparte: GitHub Desktop

VS Code ya trae todo lo necesario, así que esto es opcional. Pero si quieres un programa dedicado, con las mismas cuatro acciones y aún más a la vista, existe **GitHub Desktop** (`desktop.github.com`, gratuito, Windows y macOS).

En GitHub Desktop	Hace lo mismo que
File > Clone repository	`Git: Clone` de VS Code
La lista de la izquierda, con sus casillas	La lista de cambios con el **+**
Commit to main	El botón *Commit*
Push origin	*Sync Changes*
Fetch origin	El *pull*

Tiene una ventaja para empezar: al iniciar sesión con tu cuenta, **rellena solo** el nombre y el correo del apartado 3.4, así que te ahorras la terminal por completo.

En Linux no hay versión oficial; ahí se usa VS Code o **GitKraken**.

3.10. Lo que no hay que hacer

Esta lista está sacada de lo que falla todos los años. Ninguna de estas cosas da un aviso claro: simplemente, un día no te funciona nada y no sabes por qué.

No hagas esto	Por qué	Qué hacer en su lugar
Subir el proyecto comprimido en un ZIP	Pierdes el historial y las diferencias; es lo contrario de usar Git	Commit y push normales
Borrar la carpeta y volver a clonar porque algo falla	Pierdes lo que no hubieras subido	Lee la Parte 5 y pregunta
Clonar dentro de otra carpeta clonada	Un repositorio dentro de otro: Git se lía	Una carpeta `daw` con los repositorios al lado
Trabajar en dos ordenadores sin pull	Dos versiones que avanzan por su cuenta	Pull al empezar, push al terminar
Hacer un commit gigante al final de la semana	Nadie entiende ese commit, ni tú	Commits pequeños, uno por cosa terminada
Subir contraseñas, claves o datos personales	Quedan en el historial para siempre	`.gitignore`, y si pasa, cambia la contraseña
Dejar tu sesión de GitHub abierta en un PC del aula	Cualquiera puede subir en tu nombre	Cierra sesión en VS Code y en el navegador
Trabajar sin guardar el fichero	Git solo ve lo que está guardado en disco	`Ctrl+S` antes de preparar

Parte 4. Cosas útiles del día a día

4.1. Publicar una página web gratis

Si tu repositorio tiene un `index.html`, GitHub te lo publica en internet sin pagar nada.

1. En tu repositorio: **Settings** > en el menú de la izquierda, **Pages**.
2. En *Source*, elige **Deploy from a branch**.
3. En *Branch*, elige **main** y la carpeta **/ (root)**. **Save**.
4. Espera un par de minutos y recarga.

Tu página estará en:

`https://TU-USUARIO.github.io/NOMBRE-DEL-REPOSITORIO/`

Idea clave

Esa dirección es pública y la puedes enseñar a quien quieras. Es la forma más rápida que existe de tener tus prácticas de HTML colgadas en internet de verdad, con su enlace, para ponerlo en el currículum.

4.2. Issues: la lista de tareas del proyecto

La pestaña **Issues** sirve para anotar lo que falta por hacer y los fallos que encuentras.

1. **New issue**.
2. Título corto y claro: «El botón de enviar no valida el correo».
3. En la descripción, explica cómo reproducirlo.
4. Puedes ponerle **etiquetas** (`bug`, `enhancement`, `question`) y **asignárselo** a alguien.

Cuando lo arreglas, lo cierras con **Close issue**.

Idea clave

Aunque trabajes solo, úsalo. Es mucho mejor que una lista en el móvil: queda junto al código, con su historial, y cuando vuelvas al proyecto en tres meses sabrás por dónde ibas.

4.3. Añadir a alguien a tu repositorio

Settings > **Collaborators** > **Add people** > su nombre de usuario de GitHub. Le llega una invitación que tiene que aceptar.

A partir de ahí puede subir cambios igual que tú.

Para el examen / Ojo

En cuanto sois dos, el reparto importa. Si los dos editáis el mismo fichero a la vez, al segundo en subir le saldrá un **conflicto**: Git no sabe cuál de las dos versiones vale y os lo pregunta.
Mientras estéis en este manual, con dos normas basta:

1. **Repartíos los ficheros.** Que cada uno trabaje en los suyos.
2. **Pull antes de empezar, push al terminar.** Siempre, los dos.

Resolver conflictos, trabajar con ramas y usar pull requests está en el manual avanzado.

4.4. Buscar y guardar proyectos de otros

Qué	Para qué
La **lupa** de arriba	Buscar repositorios, código o personas

Qué	Para qué
Star (la estrella)	Guardar un proyecto para encontrarlo luego. Como un marcador
Watch	Que te avisen de los cambios de ese proyecto
Fork	Hacerte una copia propia del proyecto de otro
Botón verde **Code > Download ZIP**	Bajarte el proyecto entero a tu ordenador, sin cuenta ni nada

Idea clave

Mira el código de otros. Casi todo el software libre que usas está en GitHub: busca `vscode`, `python` o cualquier programa que te guste y entra a ver cómo está hecho por dentro. Es una de las mejores formas de aprender, y es gratis.

4.5. Tu perfil como tarjeta de visita

Hay un truco poco conocido: si creas un repositorio **con el mismo nombre que tu usuario** y le pones un README, ese README aparece en la portada de tu perfil.

1. **New repository**.
2. Nombre: exactamente tu nombre de usuario.
3. Marca *Add a README file* y créalo.
4. Edita el README con una presentación tuya.

Ahora, cuando alguien entre en `github.com/tu-usuario`, lo primero que verá es eso.

Parte 5. Cuando algo va mal

Problema	Qué pasa	Solución
«Authentication failed» al subir	La sesión ha caducado	`Ctrl+Mayús+P` > *Git: Clone* otra vez, o vuelve a iniciar sesión desde el icono de la cuenta de VS Code
«Updates were rejected»	Hay cambios en GitHub que no tienes	Haz **pull** primero y vuelve a intentarlo
He hecho commit pero no se ve en GitHub	Te falta el **push**	Pulsa *Sync Changes*
He subido un fichero con una contraseña	Borrarlo no basta: sigue en el historial	**Cambia la contraseña inmediatamente** y avisa al profesor
No encuentro el botón de nueva carpeta	No existe	Crea un fichero y escribe `carpeta/fichero.md`
El repositorio está vacío y no sé por qué	Lo creaste sin README y no has subido nada	Sube cualquier fichero
Me he equivocado en el mensaje del commit	Se puede cambiar, pero es lioso	Déjalo y haz el siguiente mejor

Para el examen / Ojo

Lo único que no tiene arreglo fácil es subir contraseñas o datos personales. Borrar el fichero no sirve: el historial guarda todas las versiones, y si el repositorio es público, cualquiera puede recuperarlo. Por eso existe el `.gitignore`. Antes de subir nada, pregúntate si ahí dentro hay algo que no quieras enseñar.

Lista de comprobación

Cuando termines el manual, repasa esto. Si puedes marcar las doce, lo tienes montado de verdad. Si alguna se te resiste, vuelve al apartado que se indica.

	Lo compruebo así	Apartado
[ ]	Entro en `github.com/mi-usuario` y sale mi perfil con mi nombre y mi foto	1.4
[ ]	Tengo la verificación en dos pasos activada	1.3
[ ]	**Tengo guardados los códigos de recuperación fuera del móvil**	1.3
[ ]	Mi repositorio tiene README y se ve formateado en la portada	2.3 y 2.5
[ ]	En la pestaña Commits hay **al menos tres** commits míos	2.11
[ ]	Ninguno de mis mensajes de commit es «cambios», «asdf» ni «update»	2.7
[ ]	Al escribir `git --version` en una terminal, contesta un número	3.3
[ ]	Tengo el repositorio clonado y VS Code abre **la carpeta del proyecto**	3.5
[ ]	El panel de Control de código fuente me lista los ficheros que toco	3.6
[ ]	He hecho un cambio en VS Code, commit, push, **y lo veo en la web**	3.6
[ ]	Sé distinguir qué hace *Commit* y qué hace *Sync Changes*	3.6
[ ]	En la carpeta no hay contraseñas ni ficheros que no deban subirse	2.3

Idea clave

La prueba definitiva, la que de verdad demuestra que lo has entendido: edita un fichero **desde la web**, guarda el commit, y luego en VS Code haz **pull**. Si el cambio aparece en tu ordenador, has cerrado el círculo completo y ya sabes trabajar con esto.

Glosario

Palabra	Qué significa
Repositorio	La carpeta del proyecto, con todo su historial
Commit	Una foto guardada del proyecto, con su mensaje
Push	Subir tus commits a GitHub
Pull	Traer a tu ordenador los cambios que haya en GitHub
Clonar	Descargar por primera vez un repositorio a tu ordenador
Stage	Marcar qué cambios entran en el próximo commit
Rama (*branch*)	Una línea de trabajo paralela. La principal se llama `main`
origin	El nombre que Git le da a la copia que está en GitHub
.git	La carpeta oculta donde se guarda el historial. No se toca
README	La portada del repositorio
.gitignore	Lista de ficheros que no se suben
Issue	Una tarea o un fallo anotado en el proyecto
Fork	Una copia de un repositorio ajeno en tu cuenta
Star	Marcar un proyecto como favorito
GitHub Pages	La función que publica tu repositorio como página web

Resumen en diez líneas

1. Git está en tu ordenador. GitHub está en internet.
2. Un repositorio es una carpeta con memoria.
3. Activa la verificación en dos pasos y **guarda los códigos de recuperación**.
4. Cada proyecto, su repositorio.
5. El README es la portada: cuídalo.
6. Commit = guardar una foto, **en tu ordenador**.
7. Push = mandarla a GitHub. **Son dos cosas distintas.**
8. Pull antes de empezar a trabajar. Push al terminar.
9. El mensaje del commit lo vas a leer tú dentro de seis meses.
10. Nunca subas contraseñas. Nunca.